



ЖИЗНЕННЫЙ ЦИКЛ ИБ

ПРАКТИЧЕСКОЕ ЗАДАНИЕ: ШИФРОВАНИЕ GPG

Выполнил: Юрий Шамрай (MIFIIB/1-й поток)

УСЛОВИЕ ПРАКТИЧЕСКОГО ЗАДАНИЯ:

- Установите GnuPG в разных системах;
- Создать пару ключей №1;
- Создать пару ключей №2 с указанием другой электронной почты;
- Зашифровать любой файл с помощью закрытого ключа №1 на открытом ключе №2;
- Расшифровать файл с помощью закрытого ключа №2.

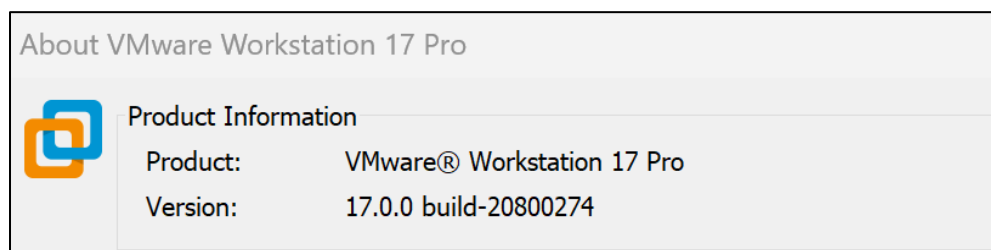
УСЛОВИЯ РЕАЛИЗАЦИИ:

- Пришлите письменный отчет в формате Word или PDF или в виде скриншотов содержимое терминала с командами и результатами их выполнения, которые были использованы в рамках практической работы.
- Одним из вариантов решения задачи, может быть, установка GnuPG в двух разных системах (на двух разных виртуальных машинах) с полноценным обменом открытыми ключами и зашифрованным файлом между ними.

ВЫПОЛНЕНИЕ

Практическое задание мы будем выполнять, на основе полного чек-листа, в виртуальной лаборатории, на базе двух виртуальных машин.

В качестве гипервизора, для их создания используем VMware Workstation 17 Pro:



со следующими параметрами:

1. **prospero-lab-01**

- OS: Ubuntu 23.04 (Lunar Lobster)
- CPU: 1CPU x 4vCORES
- RAM: 8 GB
- HDD: 30GB
- LAN: 1 Gigabit Ethernet adapter
- CD/DVD: для установки с ISO-образа
- DISPLAY: Макс. разрешение (опционально)

```
yuriy@prospero-lab-01:~$ cat /etc/os-release
PRETTY_NAME="Ubuntu 23.04"
NAME="Ubuntu"
VERSION_ID="23.04"
VERSION="23.04 (Lunar Lobster)"
```

2. **prospero-lab-02**

- OS: Ubuntu 23.04 (Lunar Lobster)
- CPU: 1CPU x 4vCORES
- RAM: 8 GB
- HDD: 30GB
- LAN: 1 Gigabit Ethernet adapter
- CD/DVD: для установки с ISO-образа
- DISPLAY: Макс. разрешение (опционально)

```
yuriy@prospero-lab-02:~$ cat /etc/os-release
PRETTY_NAME="Ubuntu 23.04"
NAME="Ubuntu"
VERSION_ID="23.04"
VERSION="23.04 (Lunar Lobster)"
```

УСТАНОВКА GnuPG

GnuPG (GNU Privacy Guard) входит в установку по умолчанию для **Ubuntu** начиная с версии **10.04 (Lucid Lynx)**. Таким образом, программа доступна в стандартной установке и для нашей версии **Ubuntu 23.04 (Lunar Lobster)**.

Если по какой-то причине **GnuPG** не установлен, то можно выполнить установку следующей командой:

```
$ sudo apt update && sudo apt install gnupg
```

Проверяем версию **GnuPG** на **Ubuntu**, выполнив следующую команду в терминале:

```
yuriy@prospero-lab-01:~$ gpg --version
gpg (GnuPG) 2.2.40
libgcrypt 1.10.1
Copyright (C) 2022 g10 Code GmbH
License GNU GPL-3.0-or-later <https://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.

Home: /home/yuriy/.gnupg
Supported algorithms:
Pubkey: RSA, ELG, DSA, ECDH, ECDSA, EDDSA
Cipher: IDEA, 3DES, CAST5, BLOWFISH, AES, AES192, AES256, TWOFISH,
        CAMELLIA128, CAMELLIA192, CAMELLIA256
Hash: SHA1, RIPEMD160, SHA256, SHA384, SHA512, SHA224
Compression: Uncompressed, ZIP, ZLIB, BZIP2
```

- **gpg (GnuPG) 2.2.40** - версия **GnuPG**, которая установлена в нашей системе;
- **libgcrypt 1.10.1** - версия библиотеки **libgcrypt**, которая используется **GnuPG** для криптографических операций;
- **Copyright (C) 2022 g10 Code GmbH** - разработан компанией **g10 Code GmbH**;
- **License GNU GPL-3.0-or-later** - распространяется под лицензией **GNU General Public License**;
- Далее информация о том, что **GnuPG** является свободным программным обеспечением, и мы можем изменять и распространять его в соответствии с условиями лицензии. А также, предупреждение о том, что программное обеспечение поставляется "как есть" без гарантий, насколько это разрешено законом.
- **Home: /home/yuriy/.gnupg** - домашняя директория **GnuPG**, где хранятся ключи и другие данные.
- **Supported algorithms** - Здесь перечислены поддерживаемые алгоритмы **GnuPG**, включая алгоритмы для шифрования (**Pubkey**), алгоритмы блочного шифрования (**Cipher**), хэширования (**Hash**) и сжатия (**Compression**).

ГЕНЕРАЦИЯ КЛЮЧЕВЫХ ПАР

После установки GnuPG нам нужно будет создать собственную пару ключей GPG, состоящую из закрытого и открытого ключей. Закрытый ключ – это наш главный ключ. Он позволяет шифровать/расшифровать наши файлы и создавать подписи, которые подписаны нашим открытым ключом.

Открытый ключ, которым мы делимся, может быть использован для проверки того, что зашифрованный файл действительно исходит от нас и был создан с использованием нашего ключа. Он также может быть использован другими для шифрования файлов для нас для расшифровки.

На каждой виртуальной машине выполним следующую команду для генерации ключевой пары:

\$ gpg --full-generate-key

Эта команда запустит мастер генерации ключей GnuPG и предложит выбрать тип ключа, длину и срок действия ключа.

Тип ключа, выбираем **(RSA and RSA)**:

```
yuriy@prospero-lab-01:~$ gpg --full-generate-key
gpg (GnuPG) 2.2.40; Copyright (C) 2022 g10 Code GmbH
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.

Please select what kind of key you want:
  (1) RSA and RSA (default)
  (2) DSA and Elgamal
  (3) DSA (sign only)
  (4) RSA (sign only)
  (14) Existing key from card
```

Длину ключа устанавливаем **(2048)**:

```
RSA keys may be between 1024 and 4096 bits long.
What keysize do you want? (3072) 2048
Requested keysize is 2048 bits
```

Срок действия устанавливаем без ограничения срока действия **(key does not expire)**:

```
Please specify how long the key should be valid.
    0 = key does not expire
<n>  = key expires in n days
<n>w = key expires in n weeks
<n>m = key expires in n months
<n>y = key expires in n years
Key is valid for? (0) 0
Key does not expire at all
```

Далее, вводим имя и адрес электронной почты:

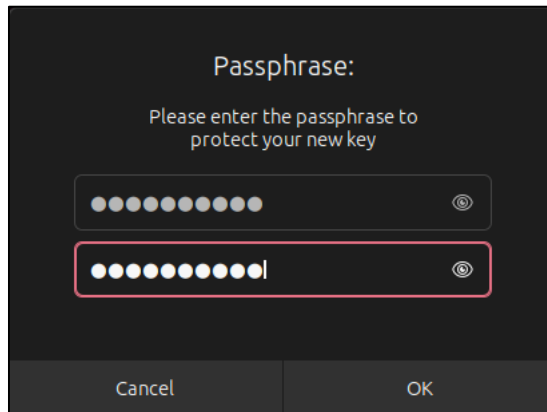
```
GnuPG needs to construct a user ID to identify your key.

Real name: Yuriy01
Email address: yuriy01@gmail.com
Comment:
You selected this USER-ID:
    "Yuriy01 <yuriy01@gmail.com>"
```

Подтверждаем, что мы ввели корректные данные **(O)kay**:

```
Change (N)ame, (C)omment, (E)mail or (O)kay/(Q)uit?
```

Прежде чем ключ будет сгенерирован, вводим парольную фразу не менее 8-ми символов:



Как только ключ будет сгенерирован мы увидим подтверждение:

```
We need to generate a lot of random bytes. It is a good idea to perform
some other action (type on the keyboard, move the mouse, utilize the
disks) during the prime generation; this gives the random number
generator a better chance to gain enough entropy.
gpg: /home/yuriy/.gnupg/trustdb.gpg: trustdb created
gpg: directory '/home/yuriy/.gnupg/openpgp-revocs.d' created
gpg: revocation certificate stored as '/home/yuriy/.gnupg/openpgp-revocs.d/851A2
0296E5D71D604901C0C4A3E853A2DF8DECC.rev'
public and secret key created and signed.

pub  rsa2048 2023-06-24 [SC]
      851A20296E5D71D604901C0C4A3E853A2DF8DECC
uid                               Yuriy01 <yuriy01@gmail.com>
sub  rsa2048 2023-06-24 [E]
```

Повторяем все шаги по генерации ключей и на второй виртуальной машине:

```
We need to generate a lot of random bytes. It is a good idea to perform
some other action (type on the keyboard, move the mouse, utilize the
disks) during the prime generation; this gives the random number
generator a better chance to gain enough entropy.
gpg: /home/yuriy/.gnupg/trustdb.gpg: trustdb created
gpg: directory '/home/yuriy/.gnupg/openpgp-revocs.d' created
gpg: revocation certificate stored as '/home/yuriy/.gnupg/openpgp-revocs.d/86BB2
A1D4CB5D6CE53E4C950F41BE0D5BB99CA7A.rev'
public and secret key created and signed.

pub  rsa2048 2023-06-24 [SC]
      86BB2A1D4CB5D6CE53E4C950F41BE0D5BB99CA7A
uid                               Yuriy02 <yuriy02@gmail.com>
sub  rsa2048 2023-06-24 [E]
```

ОТЗЫВ КЛЮЧЕЙ (ИНФО)

Ключи в GnuPG могут быть отозваны по какой-либо причине. Отзыв ключей подразумевает создание сертификата отзыва (revocation certificate), который сообщает другим пользователям, что наш ключ больше не действителен.

Чтобы создать сертификат отзыва ключа в GnuPG, необходимо выполнить следующую команду:

```
$ gpg --output revocation-certificate.asc --gen-revoke "имя_ключа"
```

Здесь "имя_ключа" — это идентификатор или отпечаток нашего ключа.

В результате будет создан файл [revocation-certificate.asc](#), содержащий сертификат отзыва ключа.

Далее сохраним этот файл в безопасном месте, так как он важен для отзыва ключа в будущем.

Чтобы отозвать ключ с использованием сертификата отзыва, выполните следующую команду:

```
$ gpg --import revocation-certificate.asc
```

После этого наш ключ будет отозван и станет недействительным.

ОБМЕН ОТКРЫТЫМИ КЛЮЧАМИ

На каждой виртуальной машине экспортируем открытый ключ и скопируем его на другую виртуальную машину.

На первой виртуальной машине **prospero-lab-01** выполним следующую команду для экспорта открытого ключа:

```
$ gpg --export -a "Yuriy01" > public_key_yuriy_01.asc
```

```
yuriy@prospero-lab-01:~$ gpg --export -a "Yuriy01" > public_key_yuriy_01.asc
yuriy@prospero-lab-01:~$ ls
Desktop  Downloads  Pictures  public_key_yuriy_01.asc  Templates
Documents Music      Public    snap                    Videos
```

На второй виртуальной машине **prospero-lab-02** выполним следующую команду для экспорта открытого ключа:

```
$ gpg --export -a "Yuriy02" > public_key_yuriy_02.asc
```

```
yuriy@prospero-lab-02:~$ gpg --export -a "Yuriy02" > public_key_yuriy_02.asc
yuriy@prospero-lab-02:~$ ls
Desktop  Downloads  Pictures  public_key_yuriy_02.asc  Templates
Documents Music      Public    snap                    Videos
```

При экспорте ключа в GnuPG мы указываем имя файла, в котором сохраняется экспортированный ключ. Если не указано конкретное место для сохранения файла, то ключ будет сохранен в текущей рабочей директории, то есть в директории, из которой была выполнена команда экспорта.

Скопируем открытый ключ с **prospero-lab-01** с использованием команды **scp** на **prospero-lab-02**:

```
$ scp ~/public_key_yuriy_01.asc yuriy@192.168.1.26:/home/yuriy/
```

```
yuriy@prospero-lab-01:~$ scp ~/public_key_yuriy_01.asc yuriy@192.168.1.26:/home/yuriy/
The authenticity of host '192.168.1.26 (192.168.1.26)' can't be established.
ED25519 key fingerprint is SHA256:QLW1CV37C09wrc82gQ8nD/U57UoL1Q4KdPjuVDgoxP0.
This key is not known by any other names
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '192.168.1.26' (ED25519) to the list of known hosts.
yuriy@192.168.1.26's password:
public_key_yuriy_01.asc                               100% 1737   720.8KB/s   00:00
```

Скопируем открытый ключ с **prospero-lab-02** с использованием команды **scp** на **prospero-lab-01**:

```
$ scp ~/public_key_yuriy_02.asc yuriy@192.168.1.22:/home/yuriy/
```

```
yuriy@prospero-lab-02:~$ scp ~/public_key_yuriy_02.asc yuriy@192.168.1.22:/home/yuriy/
The authenticity of host '192.168.1.22 (192.168.1.22)' can't be established.
ED25519 key fingerprint is SHA256:g7q5IrL4jrgndGF7Twr/6qzULoAIiE6NZqp2aWYqmZw.
This key is not known by any other names
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '192.168.1.22' (ED25519) to the list of known hosts.
yuriy@192.168.1.22's password:
public_key_yuriy_02.asc                               100% 1737   723.6KB/s   00:00
```

Проверяем на виртуальных машинах, что ключи скопировались:

```
yuriy@prospero-lab-01:~$ ls
Desktop  Downloads  Pictures  public_key_yuriy_01.asc
Documents Music      Public    public_key_yuriy_02.asc
```

```
yuriy@prospero-lab-02:~$ ls
Desktop  Downloads  Pictures  public_key_yuriy_01.asc
Documents Music      Public    public_key_yuriy_02.asc
```

Импортируем ключи:

```
yuriy@prospero-lab-01:~$ gpg --import public_key_yuriy_02.asc
gpg: key F41BE0D5BB99CA7A: public key "Yuriy02 <yuriy02@gmail.com>" imported
gpg: Total number processed: 1
gpg:          imported: 1
```

```
yuriy@prospero-lab-02:~$ gpg --import public_key_yuriy_01.asc
gpg: key 4A3E853A2DF8DECC: public key "Yuriy01 <yuriy01@gmail.com>" imported
gpg: Total number processed: 1
gpg:          imported: 1
```

Просмотр списка доступных ключей:

```
yuriy@prospero-lab-01:~$ gpg --list-keys
/home/yuriy/.gnupg/pubring.kbx
-----
pub   rsa2048 2023-06-24 [SC]
      851A20296E5D71D604901C0C4A3E853A2DF8DECC
uid           [ultimate] Yuriy01 <yuriy01@gmail.com>
sub   rsa2048 2023-06-24 [E]

pub   rsa2048 2023-06-24 [SC]
      86BB2A1D4CB5D6CE53E4C950F41BE0D5BB99CA7A
uid           [ unknown] Yuriy02 <yuriy02@gmail.com>
sub   rsa2048 2023-06-24 [E]
```

```
yuriy@prospero-lab-02:~$ gpg --list-keys
gpg: checking the trustdb
gpg: marginals needed: 3 completes needed: 1 trust model: pgp
gpg: depth: 0 valid: 1 signed: 0 trust: 0-, 0q, 0n, 0m, 0f, 1u
/home/yuriy/.gnupg/pubring.kbx
-----
pub   rsa2048 2023-06-24 [SC]
      86BB2A1D4CB5D6CE53E4C950F41BE0D5BB99CA7A
uid           [ultimate] Yuriy02 <yuriy02@gmail.com>
sub   rsa2048 2023-06-24 [E]

pub   rsa2048 2023-06-24 [SC]
      851A20296E5D71D604901C0C4A3E853A2DF8DECC
uid           [ unknown] Yuriy01 <yuriy01@gmail.com>
sub   rsa2048 2023-06-24 [E]
```

Теперь обе виртуальные машины имеют открытые ключи друг друга.

ШИФРОВАНИЕ И ДЕШИФРОВАНИЕ СООБЩЕНИЙ

На отправляющей виртуальной машине [prospero-lab-01](#) создадим файл `message.txt`:

```
yuriy@prospero-lab-01: ~
GNU nano 7.2 message01.txt *
Этот файл зашифрован с помощью GnuPG
для prospero-lab-02
```

Далее, выполним следующую команду для шифрования файла:

\$ gpg --encrypt --recipient "Yuriy02" --output encrypted_message01.txt message01.txt

```
yuriy@prospero-lab-01:~$ gpg --encrypt --recipient "Yuriy02" --output encrypted_message01.txt message01.txt
gpg: 33591AA8194BF26A: There is no assurance this key belongs to the named user

sub   rsa2048/33591AA8194BF26A 2023-06-24 Yuriy02 <yuriy02@gmail.com>
Primary key fingerprint: 86BB 2A1D 4CB5 D6CE 53E4 C950 F41B E0D5 BB99 CA7A
Subkey fingerprint: D422 FF56 780C 8149 268A A8C5 3359 1AA8 194B F26A

It is NOT certain that the key belongs to the person named
in the user ID. If you *really* know what you are doing,
you may answer the next question with yes.

Use this key anyway? (y/N) y
```

Скопируем этот файл на принимающую виртуальную машину **prospero-lab-02**:

```
yuriy@prospero-lab-01:~$ scp ~/encrypted_message01.txt yuriy@192.168.1.26:/home/yuriy/  
yuriy@192.168.1.26's password:  
encrypted_message01.txt 100% 435 209.8KB/s 00:00
```

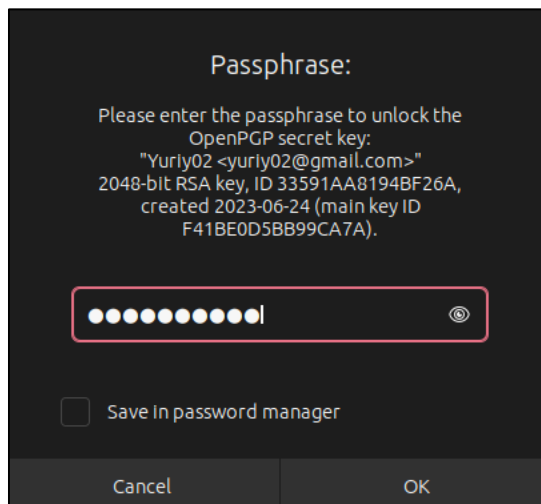
Проверяем, что файл скопировался:

```
yuriy@prospero-lab-02:~$ ls  
Desktop Downloads  
Documents encrypted_message01.txt
```

Выполним следующую команду для дешифрования сообщения:

```
$ gpg --decrypt encrypted_message01.txt
```

Вводим парольную фразу для закрытого ключа, что мы установили ранее:



A dark-themed dialog box titled "Passphrase:". It contains the text: "Please enter the passphrase to unlock the OpenPGP secret key: 'Yuriy02 <yuriy02@gmail.com>' 2048-bit RSA key, ID 33591AA8194BF26A, created 2023-06-24 (main key ID F41BE0D5BB99CA7A).". Below the text is a password input field with 10 dots and a toggle icon. At the bottom left is a checkbox labeled "Save In password manager". At the bottom are "Cancel" and "OK" buttons.

Файл был успешно дешифрован:

```
yuriy@prospero-lab-02:~$ gpg --decrypt encrypted_message01.txt  
gpg: encrypted with 2048-bit RSA key, ID 33591AA8194BF26A, created 2023-06-24  
"Yuriy02 <yuriy02@gmail.com>"  
Этот файл зашифрован с помощью GnuPG  
для prospero-lab-02  
yuriy@prospero-lab-02:~$ ls  
Desktop Downloads message.txt Pictures public_key_yuriy_01.asc snap Videos  
Documents encrypted_message01.txt Music Public public_key_yuriy_02.asc Templates  
yuriy@prospero-lab-02:~$
```

Теперь мы можем обмениваться зашифрованными сообщениями между виртуальными машинами, используя GnuPG.

ПРАКТИЧЕСКАЯ РАБОТА ЗАВЕРШЕНА!